Replay Resistant Telemetry for Digital Twin Synchronisation

Ramiz Aliguliyev

[1]University Of Sheffield

Problem & Motivation

This poster addresses the replay-attack risk identified in Section V-B of our group SoK paper, where we showed that DT telemetry channels rarely authenticate originality and that the twin occupies the same observational role as the SCADA monitor in the Stuxnet case. Digital Twins rely on continuous real-time data collection from sensors, PLCs, and industrial controllers to stay synchronized with the physical system. This data is often carried over industrial protocols such as MQTT, OPC-UA, and ModbusTCP, which may not provide strong message-level authentication or freshness guarantees. If an attacker captures legitimate telemetry and retransmits it later, the Digital Twin may continue to see a normal operating state even when the real plant is faulty or under attack.[5]

This is a replay attack. The attacker does not need to fully understand the Digital Twin or modify the data itself. Instead, they can simply reuse previously recorded valid "healthy" messages to hide abnormal behaviour. This reflects the same general network-layer pattern seen in Stuxnet, where normal-looking data was used to mislead monitoring systems while the real process was being manipulated. In a Digital Twin setting, the risk is especially serious because the twin occupies the same observational role, it receives telemetry, represents system state, and feeds that state into operator dashboards, anomaly detection, and predictive maintenance services. If the telemetry is replayed, the malicious data can spread across the whole architecture. [4]

Threat Model

This threat model focuses on the communication channel that carries telemetry from the physical system to the Digital Twin. The attacker is not assumed to control the sensor, PLC, or Twin Engine. Instead, the attacker sits on the network path and abuses the fact that old legitimate telemetry may still look valid to the twin. By replaying previously captured "healthy" messages during a real fault, the attacker can make the Digital Twin show an outdated and misleading version of the physical process.[3]

- **Attacker capability:** on-path network attacker able to capture and replay telemetry packets.
- **Attacker knowledge:** protocol format is known, but the internal Digital Twin model is treated as a black box.
- **Attacker goal:** desynchronise the Digital Twin from the real system by replaying previously captured "healthy" telemetry during a fault.
- **Assets protected:** integrity and freshness of the sensor-to-twin telemetry stream.
- **Out of scope:** physical sensor compromise, AI/ML layer attacks, insider key compromise

References

[1] P. Syverson, "A Taxonomy of Replay Attacks," *Proceedings of the IEEE Computer Security Foundations Workshop*, 1994.

[2] Y. Mo and B. Sinopoli, "Secure Control Against Replay Attacks," *47th Annual Allerton Conference on Communication, Control, and Computing*, 2009.

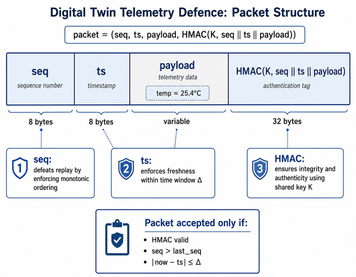
[3] M. Pooyandeh and I. Sohn, "A time-stamp attack on digital twinbased lithium-ion battery monitoring for electric vehicles," 2024 International Conference on Artificial Intelligence in Information and Communication (ICAIIC). IEEE, 2024.

[4] S. Karnouskos, "Stuxnet Worm Impact on Industrial Cyber-Physical System Security," *37th Annual Conference of the IEEE Industrial Electronics Society (IECON)*, 2011.

[5] E. Karaarslan and M. Babiker, "Digital Twin Security Threats and Countermeasures: An Introduction," *2021 International Conference on Information Security and Cryptology (ISCTURKEY)*, pp. 7–11, IEEE, 2021.

Proposed Defence

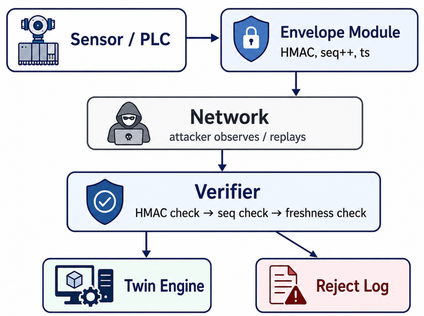
Each data sample is wrapped in an authenticated envelope at the sender:



Here, payload is the actual data value, such as temperature or pressure. The seq field is a sequence number that increases for every new packet, and ts is the timestamp showing when the packet was created. The final field is an HMAC generated hash using the shared secret key K.

The purpose of the HMAC is to prove that the packet was created by a trusted sender and that its contents were not modified in transit. However, HMAC alone does not stop replay attacks, because an old packet can still have a valid HMAC. Therefore, the verifier also checks that the sequence number is newer than the last accepted packet and that the timestamp is still within an allowed freshness window.[1]

System Architecture



The defence is placed at the Digital Twin input boundary. Each sensor or PLC message is wrapped by the Envelope Module with HMAC, seq, and ts before being sent over the network. Since an attacker may observe and replay packets in transit, the Verifier checks every received message before it reaches the Twin Engine. Packets are accepted only if the HMAC is valid, the sequence number is newer than the last accepted packet, and the timestamp is within the freshness window. Failed packets are dropped and logged.[2]

Trust Assumptions

Assumption	Justification
Shared key K is securely provisioned before deployment	Required so the sender can generate and the verifier can validate the HMAC tag.
Sender and verifier clocks are loosely synchronised	Needed for timestamp freshness checks; small clock drift is tolerated by the freshness window.
The verifier is uncompromised	The verifier forms the trusted boundary before data is accepted by the Digital Twin engine.
Per-sender `last_seq` is persisted across restarts	Prevents reset-replay, where old packets are accepted after verifier restart.
Physical sensor compromise and key theft are out of scope	The defence targets network-level replay, not malicious sensors or stolen cryptographic keys.

Experiment

A Python prototype is used to simulate a sensor to Digital Twin telemetry stream under replay attack. The testbed generates 100 telemetry packets in total. During the first 60 packets, the plant is in a normal state and sends healthy temperature values around 25°C. From packet 60, a simulated fault begins and the real plant temperature increases progressively. The attacker captures 10 healthy packets from the normal period and replays them across the 40-packet fault window, cycling through the captured set, so the Digital Twin would observe a continuous 'healthy' stream in place of the real fault telemetry. This produces 40 replay attempts in total.

Scenario	Accepted	Rejected	Replay Acceptance rate	False Rejects	Avg latency
S1 Normal Traffic	100	0	N/A	0%	N/A
S2 Replay Attack, no defence	100	0	100%	N/A	N/A
S3- HMAC defence	60	40	0.0%	0.0%	2.56µs

Table 1. Experiment Results.

In this configuration the sequence-number check is the first to fail for every replay attempt; the freshness window would also reject them, but the seq check makes the defence robust even when an attacker chooses captured packets close in time.

Drawbacks and Trade-offs:

- **Overhead:** each packet adds roughly 48 bytes (HMAC tag, seq, ts) and 2.56 µs verification latency (Table 1) — negligible at typical sensor rates, but non-trivial at kHz update rates or on legacy PLCs without hardware-accelerated SHA-256.
- **Key management and clock-sync tension:** the shared key K must be securely provisioned and rotated across all senders; the freshness window trades replay protection against tolerance for legitimate jitter, so tighter windows catch more replays but raise false-reject rates under NTP drift.
- **Deployment feasibility:** the defence requires modifying both the sender (envelope module) and the verifier, which is straightforward for greenfield IIoT but harder for brownfield industrial systems with legacy PLCs that cannot easily be re-flashed.
- **Threats to validity:** single-host simulation, simplified attacker model, and synthetic temperature telemetry; production networks may introduce jitter, packet loss, and clock drift.